Synchronization Lock Taxonomy

To maximize execution performance under heavy multimodal payloads, the system implements multi-worker parallel thread pools. To ensure absolute database integrity, precise visual log formatting, and stable hardware execution across these parallel steps, the pipeline utilizes thread synchronization locks.

These locks are categorized by their logical responsibilities and mapped directly to the conceptual steps of the pipeline workflow:

Bounding Box Discovery & Detail Subdivision Steps

During the visual object discovery and regional crop subdivision steps, parallel agent threads execute concurrently across different document pages. To prevent write collisions and coordinate gaps over shared on-disk files, the following locks are utilized: - **Drafts Queue Lock:** Serializes parallel discovery and subdivision agents from concurrently appending coordinate proposals to the persistent drafts queue, ensuring that coordinates are written to disk atomically one thread at a time. - **Census Checklist Lock:** Ensures that completing, updating, or rolling back individual checklist items inside page checklists remains atomic per page and fully safe against concurrent overwrites.

Programmatic Verification Steps

During the macro-level and micro-level programmatic verification steps, separate concurrent verifier threads evaluate draft coordinates against spatial layout rules. To protect catalogue promotion, the following locks are enforced: - **Promoted Regions Lock:** Prevents concurrent verifier threads from overlapping and corrupting data when writing promoted, verified regions to the master catalog, securing a single serialization channel. - **Verification Tracker Lock:** Safely synchronizes internal task-state checkpoints during multi-round verification sequences.

Visual Region Captioning & Asset Layer Assignment Steps

During the asset captioning and layer-assigner steps, parallel worker threads compile textual annotations and propose ontology layers. To serialize metadata and counter states, the following locks are applied: - **Master Inventory Lock:** Serializes concurrent layer-assignment threads to ensure that updates are written atomically to the master asset inventory without file corruption. - **Summary Counter Locks:** Protects and accurately increments real-time counter metrics (processed, skipped, errors) across parallel worker threads, guaranteeing correct final summaries on the screen terminal.

Search Index Building Steps

During the hybrid decoupled index-building steps (including visual captions indexing, visual crops indexing, and prose text chunks indexing), the pipeline processes translations, captions, and crop images concurrently. To serialize local machine learning computations on the GPU or CPU, the following locks are active: - **Text Embedding Lock:** Serializes local batched text embedding passes (such as BGE-M3 model forward runs). This prevents shared-model singleton conflicts, Metal driver queue collisions, and unified memory Out-of-Memory (OOM) spikes on Apple Silicon and CUDA hardware. - **Image Embedding Lock:** Serializes local batched visual crop embedding passes (such as SigLIP model 4D tensor forward runs), ensuring absolute hardware stability and bounded memory allocations.

Cross-Cutting Pipeline Observability

To prevent visual character stream corruption and keep log files perfectly structured, the following locks are applied globally across all standard parallel steps and agentic loops: - **Console Display Locks:** Synchronizes character streams during parallel thread executions to prevent garbled, interleaved progress bars and console alert blocks on the screen terminal. - **Session Logging Locks:** Serializes multi-worker logging streams, ensuring that log records are written and flushed atomically to the JSONL log files without line interleaving.